

Jira Developer Reports

Analysis, Comment Responses & Build Recommendations

*Prepared July 6, 2026 • Responds to all reviewer comments in
Jira_Developer_Reports_Definition • Cross-referenced against the current Azure-hosted
reporting app*

1. Executive Summary

The 18-report catalog is well conceived but should not be built as 18 separate screens. Roughly a third of the reports share the same underlying data and question, and several depend on process decisions (due dates, start dates, sprint boards, a blocked convention, worklog policy) that have not been made yet. This document answers every open comment, then recommends consolidating the catalog into seven focused screens, sequenced so that nothing is built before its data foundation exists.

Good news on effort: the Azure-hosted app already contains most of the hard machinery. The changelog analytics engine (validated against real LIFEDATAV2 tickets), the status-classification config, cycle-time and time-in-status math, transition-author attribution, per-developer pages, an executive dashboard, and Excel/CSV/JSON exports all exist. Most of the 18 reports are new views over data the app already fetches, not new systems.

The three highest-leverage moves: (1) turn Reports 1, 4, 9 and 13 into a single per-ticket End-of-Day Checklist — Kevin’s checklist idea is both doable and the best product idea in the comments; (2) merge the four “something is wrong” reports (Silent, Stuck, Overdue, Blocked) into one Attention Board with reason tags; (3) add the team-level summary layer Kevin requested so meetings discuss trends, not individuals.

Two reports should be deferred or repurposed rather than built as specified: Worklog Completeness (only meaningful if a worklog policy is adopted first) and Tickets Without Estimate (story points are not currently populated; fold it into a broader Planning Hygiene check with the new date rules). Status Change Without Comment should be narrowed to key transitions only — as specified it will be noisy and will feel like surveillance.

2. Where We Stand: the Current Azure App

From the prior build session, the deployed app (Python/Flask, read-only Jira REST + Agile API) already includes:

- Changelog analytics engine — per-ticket status timeline, stage durations, pause and reopen detection; validated against real tickets (e.g. a 29.3-day cycle time and a 28-day pause computed correctly).
- Status mapping config (config.py) — your real workflow statuses (“In Progress / Start Investigation”, “Development / In Design”, “Ready for QA (QA Env)”, “Reopen”, “Pause Investigation”, etc.) mapped to logical stages. This is the answer to the “what counts as active?” comment — it just needs to be agreed, completed, and surfaced in the UI.
- Reports live today: Executive Dashboard (/exec), Daily Work Movement, Developer Productivity (output, reopened count, quality score, credited to the transition author — not just assignee), QA Productivity (rejection rate), Status Duration Analysis, Release Readiness (fixVersion-based), Individual Activity, Workload (open assigned per developer), and Time in Status with 24h/7d/30d/custom windows, in-window vs lifetime modes, and a changed-in-window filter.
- Sprint Health is coded but dormant — it needs board IDs and, per Braden’s comment, real sprint boards set up in Jira first.
- Exports: Excel, CSV, JSON feed; 5-minute cache; Dockerfile, tests, CONTRIBUTING guide.

What the app does not yet ingest: comments and worklogs. Several of the 18 reports (Daily Activity detail, EOD Discipline, Handoff Quality, Status-Change-Without-Comment, Worklog Completeness, Timeline) need those two data sources added to the fetch layer. That is the single biggest technical delta between the document and the deployed app.

#	Report in document	Azure app status	Notes
1	Daily Developer Activity	Partial	Status changes covered by Daily Work Movement; comments/worklogs not ingested yet
2	Silent / No Daily Update	Not built	Simple query once “active” statuses agreed
3	Multiple Active Tickets	Partial	Workload view shows open assigned; no violation rule yet
4	End-of-Day Discipline	Not built	Becomes the EOD Checklist (see §3, C0)
5	Ready for QA Contribution	Partial	Developer Productivity already credits transition authors
6	Returned from QA / Reopened	Partial	Reopen/rejection detection exists in analytics engine
7	Cycle Time by Developer	Built	Core of the current app; validated on real tickets
8	Stuck Ticket Aging	Partial	Time-in-status exists; thresholds and disposition tracking do not
9	Worklog Completeness	Not built	Worklogs not ingested; policy decision needed first
10	Tickets Without Estimate	Not built	Story points not populated in the instance
11	Overdue Tickets	Not built	Depends on due-date enforcement decision
12	Developer Handoff Quality	Not built	Needs comment ingestion + keyword config
13	Status Change Without Comment	Not built	Recommend narrowing scope before building
14	Blocked Tickets	Not built	Needs a standardized blocked signal first
15	Sprint Commitment vs Completion	Coded, dormant	Blocked on sprint board setup + board IDs
16	Ticket Movement Timeline	Partial	Engine computes the timeline; no UI, no comments/worklogs
17	Developer Focus	Not built	Straightforward once activity feed exists
18	Bug Fix Quality	Partial	Reopened counts and quality score exist per developer

3. Answers to Every Comment

Each comment from the review, with a direct answer and the recommended action.

C0 — Kevin, on Daily Developer Activity

“Can we make this like a checklist? ... show the list of all things that should be completed related to a ticket ... a developer can pull this report for the day, find things not completed, and be sure they are done before ending their day. Is this doable?”

Yes, fully doable, and it is the best idea in the review. Everything on a sensible checklist is already computable from data the app fetches (or will, once comments/worklogs are ingested): status is current, a comment was added today, a worklog was logged today, estimate/due date present, blocked state flagged, handoff notes present if the ticket moved to Ready for QA.

This collapses Reports 1, 4, 9 and 13 into one product: a “My Day” view listing each developer’s active tickets as checklist rows with green/red items. The developer runs it at 4:30pm, fixes the red items, and ends the day clean. The admin view of the same engine is the compliance summary (Report 4).

Recommendation: make this the developer-facing home page of the app. Checklist items should be configurable (per project) so the rules can evolve without code changes.

C1 — Kevin, on Silent Tickets

“We need rules about what needs to be done with these. Maybe a start date required for each ... which they can move, but the start date should never be in the past? Could help developers adjust priorities.”

Agree, with one refinement: for a ticket already in progress, its start date will legitimately be in the past. The workable rule is: every ticket in an active status must have a start date; for not-yet-started tickets the planned start date must be today or later; when a planned date passes without work starting, the developer must either start it or move the date.

Each move is recorded in the changelog, which gives us a free and telling metric: Reschedule Count (and total days pushed). A ticket rescheduled five times is a prioritization problem, not a developer problem — exactly the coaching signal you want.

Enforcement: Jira can require fields on transitions in company-managed projects (workflow validators); team-managed projects need an Automation rule instead. Simplest path: let the app flag violations (missing/stale start dates) on the Attention Board and in the EOD checklist rather than hard-blocking in Jira on day one.

C2 — Kevin + Braden, on Multiple Active Tickets

“What statuses count as ‘active’? Tanvir has many tickets in Ready for QA or QA review completed that never move beyond that.” / “If there are more statuses than ‘In Progress’ that mean active, I will need a list of those.”

This is the foundational decision for the whole catalog, and the app already has the structure for it: config.py maps every workflow status to a logical bucket. Proposed classification for LIFEDATAV2 (to be confirmed against the live workflow): Active-Dev = In Progress / Start Investigation, Development / In Design (and any other development statuses); QA-Stage = Ready for QA (QA Env), In QA Testing (QA Env), QA Review Completed; Paused = Pause Investigation; Rework = Reopen; Done = Development Completed / Closed / Released.

The Tanvir case is the key insight: tickets parked in Ready for QA or QA Review Completed are not “active dev” — they should not count against the one-active-ticket rule — but they are stuck, which is

Report 8's job. The fix is per-status aging thresholds: e.g. anything in QA Review Completed longer than 2 days appears on the Attention Board. That surfaces the "never moves beyond QA" pattern explicitly instead of hiding it in the active-count rule.

Recommendation: build a small admin settings screen where statuses are dragged into buckets and thresholds set per status. Then workflow changes never require a deploy, and the "list of active statuses" question has a permanent, visible answer.

C3 — Kevin, on Cycle Time ("where delays occur")

"How does it show this? A feature like this could be important."

Exactly as Braden replied: from changelog timestamps. Every status change is timestamped, so for each ticket we compute time spent in every stage: coding time, waiting-for-QA time, QA time, reopen loops, paused time. The delay is wherever the biggest number sits — and aggregated across tickets, the bottleneck stage is obvious (e.g. "tickets average 1.5 days in development but 4.2 days waiting in Ready for QA").

This is already built: the Status Duration Analysis and Time in Status reports in the Azure app do precisely this, validated against real tickets. The improvement is presentation: a stacked horizontal bar per ticket (one color per stage) makes the longest segment jump out visually, and a per-stage median chart shows the team-level bottleneck.

C4 — Kevin, on Stuck Ticket Aging

"These tickets need to be forced either to a designated backlog or given a date when they will be worked on next."

A report cannot force, but it can make the rule enforceable: add a Disposition Required state. Any ticket past its aging threshold appears on the Attention Board flagged "needs disposition", and it stays flagged until one of two things happens (both visible in the changelog): it moves to Backlog, or it gets a future start date (ties into C1).

The report then tracks disposition compliance — how many flagged tickets were dispositioned within 48 hours — which is a clean team metric for the weekly meeting.

For actual enforcement, use Jira Automation (e.g. after N days in status: add a flag, comment, and notify the assignee) rather than giving the reporting app write access. Keep the app read-only; it is much safer and keeps the tool trusted.

C5 — Kevin, on Tickets Without Estimate

"Is this all about a time estimate in hours? I'm not sure how helpful this is but we can discuss."

It covers both the Jira original-estimate (hours) field and story points — and your skepticism is warranted: the build session found no populated story-points field in your instance, and hour estimates only matter if the team actually estimates. As specified, this report would just list every ticket.

Recommendation: do not build it as written. Repurpose it as a Planning Hygiene check inside the EOD checklist and Attention Board: does an active ticket have a start date (C1), a due date (C6), and — only if you adopt estimation later — an estimate? Dates are the fields you actually want to enforce; estimates can be added to the same check whenever the team starts using them.

C6 — Kevin, on Overdue Tickets

"How helpful is this if due dates are not usually required? I'm thinking we should start enforcing due dates ... requiring them helps us understand prioritization and how much tickets are delayed from the original commitment."

Agree on both counts: the report is near-useless without due dates, and enforcing them is the right call. Require a due date when a ticket enters an active status (validator or Automation rule, same options as C1).

The part that makes this genuinely insightful: Jira's changelog records every due-date change. So the report can show Original Due Date vs Current Due Date, number of pushes, and total slip days — “how much tickets are delayed from the original commitment” is directly computable. A ticket that is technically not overdue but has been pushed four times is often the more important signal.

Combined with the start-date rule (C1), dates become the planning spine: planned start → actual start (changelog) → original due → slips → actual done. That one chain answers most prioritization and delay questions leadership will ask.

C7 — Kevin, on Blocked Tickets

“What are the metrics we use to accomplish this?”

Detection signals available in Jira, in order of reliability: (1) a dedicated Blocked/On Hold status; (2) Jira's built-in Flagged field (“Flagged as impediment” — recorded in the changelog, so we can compute exactly how long a ticket has been flagged); (3) labels like blocked/waiting/dependency; (4) issue links of type “is blocked by”; (5) keyword scan of recent comments (weakest — use only as a hint).

Metrics once detected: blocked count, days blocked (from the changelog timestamp of the flag/status/label), days since last update while blocked, who/what it is waiting on (from the link or a required “blocked reason” comment), and blocked-time as a share of cycle time (the analytics engine already computes paused time this way).

The honest prerequisite: pick one convention. If detection relies on developers maybe-using labels, the report is guesswork. Recommendation: standardize on Jira's Flagged field + a required reason comment; it needs no workflow change and is fully changelog-tracked.

C8 — Kevin + Braden, on Sprint Commitment vs Completion

“I like this in concept. I want to better understand how we achieve this.” / “First we need to set up sprint boards in Jira and get our Release boards updated ... Right?”

Braden is right: real scrum boards with started/ended sprints are the prerequisite. Mechanics once they exist: the Agile API lists each sprint with start/end dates; commitment = issues in the sprint at the moment it starts; scope creep = issues whose Sprint field changed mid-sprint (changelog shows exactly when each ticket entered/left); completed = issues Done by sprint end; the remainder is carryover.

Good news: this report is already coded in the Azure app (Sprint Health) — it is waiting only on board IDs and real sprint usage. Story points are optional; ticket counts work fine until estimation is adopted (see C5).

Interim option: until sprint boards are live, the fixVersion-based Release Readiness report (already built) provides a commitment-vs-completion view against releases instead of sprints — the build session confirmed fixVersions like “Web Feature Release 0.12.0” are populated in your instance.

C9 — Kevin, on Ticket Movement Timeline

“I'm not sure how this works and how it achieves the stated objective. Need to discuss and understand more.”

Think of it as the audit trail for one ticket, not a monitoring report. It merges three streams the API provides — the changelog (every status/assignee/field change, who and when), all comments, and all worklogs — into a single chronological timeline.

The objective it serves: when someone asks “why did LIFEDATAV2-1234 take six weeks?”, the timeline shows the answer at a glance — e.g. two days of development, then 14 days sitting in Ready for QA with no activity, then a reopen loop, then a 10-day pause with no comment. Inactivity gaps and repeated back-transitions are the tell-tale patterns.

It costs almost nothing to build (the analytics engine already computes the status timeline per ticket; it needs comments/worklogs merged in and a UI) and it is the tool you reach for in every escalation and dispute. Keep it.

C10 — Kevin, on Developer Focus

“I like it”

Noted — keep as specified. One design note: it overlaps with Report 3 (both measure focus). Report 3 is the rule (“do not hold multiple active tickets”); Report 17 is the behavior (“how scattered was the day actually”). Present them together on one Focus page: the violation list on top, the touches-per-day trend below.

C11 — Kevin + Braden, on Notes for Admins

“We should generate summary reports at the team level ... discussion points during full team meetings so we are not calling out a specific individual publicly.” / “Love this!”

Strong agreement, and this should shape the app’s structure: every metric gets a team-level aggregate view, and that is what goes on screen in meetings. Examples: % of active tickets with an EOD update (not who missed), median cycle time trend, QA return rate trend, blocked count and average days blocked, disposition compliance.

Concrete feature: a Meeting Mode toggle — aggregates only, names hidden or replaced with distribution charts (“3 tickets aged >7 days” not “Tanvir’s tickets”), week-over-week deltas, large type for screen-sharing. Individual drill-downs stay available to leads for private 1:1 coaching and to each developer for their own data.

The Azure app’s /exec dashboard is the seed of this layer; it needs the discipline metrics added and the anonymized presentation mode.

4. Decisions Needed Before Building (in order)

Most of the catalog's accuracy depends on process agreements, not code. These eight decisions unblock everything; each maps to comments above.

#	Decision	Recommendation	Unblocks
1	Status classification (active / QA / paused / rework / done) + per-status aging thresholds	Confirm the draft in §3-C2 against the live workflow; make it editable in an admin UI	Reports 2, 3, 8; every metric
2	Blocked convention	Jira Flagged field + required reason comment	Report 14
3	Due-date policy	Require on entry to active status; track slips from changelog	Report 11
4	Start-date rule	Require planned start; moving it is allowed and counted	Report 2 rules, C1
5	Sprint boards + release boards	Set up scrum boards; capture board IDs (JIRA_BOARD_IDS)	Report 15 (already coded)
6	Worklog policy	Decide whether time logging is required at all; if not, drop Report 9	Reports 1, 4, 9
7	Story points / estimation	Defer; use ticket counts until the team estimates	Reports 10, 15 point metrics
8	Usage norms	Team aggregates in meetings; individual data for self-view and private coaching only	Adoption and trust

5. Report-by-Report Analysis

Verdict key: KEEP (build/retain as core), MERGE (fold into a combined screen), CONDITIONAL (build only after its prerequisite decision), DEFER (park until the team practice exists), NARROW (build a reduced version).

1. Daily Developer Activity

Verdict: KEEP — transform

Purpose: the raw activity trail — who did what, on which tickets, when.

Implementation: mostly exists (Daily Work Movement covers status changes); add comment and worklog ingestion to complete it. Per C0, evolve the developer-facing version into the EOD Checklist — that is the form in which developers will actually use it daily. The raw chronological feed remains as the admin/audit lens.

Value: high. This is the backbone feed several other reports derive from.

2. Silent / No Daily Update Tickets

Verdict: MERGE into Attention Board

Purpose: active tickets going quiet.

Implementation: trivial once “active” is defined (decision 1) — active status + updated-date older than threshold. Add C1’s rules: silent tickets demand a start-date move or an update.

Value: high as a signal; too thin to be its own screen. It becomes one reason-tag (“Silent 3d”) on the combined Attention Board.

3. Multiple Active Tickets Violation

Verdict: KEEP

Purpose: enforce the one-active-ticket focus rule.

Implementation: count tickets per developer in Active-Dev statuses only (per C2 — QA-stage tickets excluded, handled by aging instead). The Workload view already shows per-developer open tickets; add the rule and violation highlighting.

Value: high, cheap. Pair on one page with Report 17.

4. End-of-Day Discipline

Verdict: MERGE into EOD Checklist

Purpose: did each active ticket get an update/comment/worklog today?

Implementation: same engine as Report 1 + checklist config. The Yes/No columns in the spec become checklist items. Depends on comment/worklog ingestion.

Value: high — as the admin summary of the checklist, not a separate report.

5. Ready for QA Contribution

Verdict: KEEP

Purpose: throughput measured by who actually moved work to Ready for QA — robust to assignee changing at handoff.

Implementation: transition-author attribution already built (Developer Productivity credits the changelog author). Needs the dedicated handoff-feed view with previous/current status and current assignee.

Value: high — this is the fairest throughput measure in the catalog.

6. Returned from QA / Reopened

Verdict: KEEP

Purpose: rework signal — QA → dev back-transitions.

Implementation: the analytics engine already detects reopens/rejections; add the report view with from/to status and the return reason (require a comment on the Reopen transition — a one-line Jira workflow/Automation change that makes this report far more useful).

Value: high — the core quality signal, and it feeds Report 18.

7. Cycle Time by Developer

Verdict: KEEP — built

Purpose: dev-start → Ready for QA → done durations.

Implementation: done and validated in the Azure app. Improvements: stage-breakdown visualization (see C3), medians/p85 rather than averages (one 28-day pause destroys an average), and an explicit policy for reopened tickets (currently first-entry; consider counting rework loops separately).

Value: high — the flagship flow metric.

8. Stuck Ticket Aging

Verdict: KEEP — extend

Purpose: unfinished tickets ranked by time in current status.

Implementation: time-in-status math exists. Add per-status thresholds (decision 1) so QA-limbo statuses get tight limits (the Tanvir case), plus C4's disposition tracking (backlog or future start date, with a compliance metric).

Value: high. Surfaces on the Attention Board with reason "Aging".

9. Worklog Completeness

Verdict: DEFER / CONDITIONAL

Purpose: time-logging discipline.

Implementation: easy once worklogs are ingested — but pointless unless the team is actually required to log time (decision 6). If the real question is "where did effort go", stage durations from the changelog answer it without any manual logging.

Value: low today. Revisit only if a worklog policy is adopted; then it appears as one EOD-checklist item rather than a standalone report.

10. Tickets Without Estimate

Verdict: DEFER / repurpose

Purpose: planning hygiene — missing estimates.

Implementation: story points are not populated in the instance and hour-estimates are not practiced (per C5). Repurpose into a Planning Hygiene check (start date + due date present) inside the checklist and Attention Board; add estimates to the same check if estimation is ever adopted.

Value: low as specified; the repurposed date-hygiene version is high value.

11. Overdue Tickets

Verdict: CONDITIONAL — with slip tracking

Purpose: delivery-risk from missed due dates.

Implementation: only after due dates become required (decision 3). Build it with the changelog-based slip metrics from C6 (original vs current due date, pushes, slip days) — that upgrade turns a naive list into the commitment-tracking report Kevin wants.

Value: high once the policy exists; near-zero before.

12. Developer Handoff Quality

Verdict: KEEP — simplified

Purpose: are Ready-for-QA tickets testable (notes, PR link)?

Implementation: needs comment ingestion. Start with two robust checks: a comment by the developer within N hours before handoff, and a PR/build link present (Jira dev-panel links or URL pattern in comments). Keyword-based “testing note” scoring is brittle — defer it, or better, put a handoff template checklist in the Jira ticket itself and just check completion.

Value: medium-high; directly attacks QA back-and-forth. Score it Pass/Needs-info only — avoid decimal “quality scores” on people.

13. Status Change Without Comment

Verdict: NARROW

Purpose: discourage silent transitions.

Implementation possible as specified, but flagging every commentless transition is noisy (many transitions are legitimately self-explanatory) and will read as surveillance. Narrow to transitions where a missing explanation actually hurts: into Reopen, into Blocked/Paused, and backward moves. Better still, require the comment in the workflow for those transitions and let Jira enforce it.

Value: low as specified; medium narrowed. Mostly absorbed by the EOD checklist and Report 12.

14. Blocked Tickets

Verdict: CONDITIONAL

Purpose: blocker visibility and removal.

Implementation: after the blocked convention is standardized (decision 2, Flagged field recommended). Metrics per C7: days blocked from changelog, waiting-on, blocked share of cycle time. Surfaces on the Attention Board with reason “Blocked”.

Value: high once the convention exists.

15. Sprint Commitment vs Completion

Verdict: KEEP — blocked on boards

Purpose: commitment reliability, scope creep, carryover.

Implementation: already coded (Sprint Health); prerequisite is real sprint boards + board IDs (Braden's comment). Use ticket counts until story points exist. Until then, Release Readiness (built, fixVersion-based) is the interim commitment view.

Value: high once sprints are practiced; do not fake it before then.

16. Ticket Movement Timeline

Verdict: KEEP

Purpose: single-ticket forensic investigation (see C9).

Implementation: analytics engine already computes the status timeline; merge in comments/worklogs and build the timeline UI with inactivity-gap highlighting.

Value: high per use, used rarely — the escalation tool. Cheap to finish.

17. Developer Focus

Verdict: KEEP

Purpose: context-switching — distinct tickets touched per day (Kevin approves, C10).

Implementation: straightforward aggregation over the activity feed once comments/worklogs are ingested. Present with Report 3 on one Focus page.

Value: medium-high; watch that it is read as a workload-shape signal, not a productivity score.

18. Bug Fix Quality

Verdict: KEEP

Purpose: bug throughput, return rate, resolution time per developer.

Implementation: reopened counts and quality scoring exist in Developer Productivity; add the bug-only lens and average resolution hours. Attribute by who developed the fix (transition author), not current assignee. Use medians; small counts make rates volatile — show the raw numbers alongside.

Value: high at team level; use individual view for coaching only (C11).

6. Consolidation: 18 Reports → 7 Screens

Eighteen menu items guarantees most go unused. The same content organizes into seven screens matching how people actually work (and mirroring the document's own §6 operating process):

Screen	Absorbs	What it is
1. My Day (developer home)	1, 4, 9, 13	Per-ticket EOD checklist: status current, comment today, dates present, handoff notes if moved to QA. Run before ending the day.
2. Attention Board (lead home)	2, 8, 11, 14	Every ticket needing intervention, one row each, with reason tags: Silent 3d / Aging 9d / Overdue 2d / Blocked / Needs disposition. Sorted by severity.
3. QA Handoff	5, 6, 12	Handoff feed (who moved what to Ready for QA), handoff-quality checks, and returned-from-QA list with reasons.
4. Flow Analytics	7, 17, 3	Cycle time with stage breakdown, time-in-status (built), focus/context-switching, active-ticket rule violations.
5. Quality	18, 6 (trends)	Bug fix quality, return-rate trends, reopen loops.
6. Sprint & Planning	15, 10, 11 (trends)	Commitment vs completion (once boards exist; Release Readiness interim), planning hygiene, due-date slip trends.
7. Ticket Investigator	16	Enter an issue key, get the full forensic timeline.

Above all seven sits the Team Trends layer (C11): the /exec dashboard extended with discipline aggregates and a Meeting Mode. Nothing from the original catalog is lost — every output column in the document lands on one of these screens.

7. UI Recommendations

Structure and navigation

- Role-based landing: developers open to My Day; leads open to the Attention Board; the exec dashboard is the meeting entry point. One codebase, three front doors.
- Global persistent filter bar (project, developer, date range) that follows the user across screens, with saved views ("My team, last 7 days"). The document's common-input-parameters section becomes one UI element instead of 18 forms.
- Group navigation by purpose (Daily / Attention / QA / Flow / Quality / Planning / Investigate), not by report number.

Key components

- Checklist rows with traffic-light chips (green check / red x / gray n-a) per item; click a row to expand a ticket drawer with its mini-timeline and a deep link into Jira. Every issue key everywhere links to Jira.
- Attention Board as cards or a dense table with colored reason chips; multiple reasons stack on one ticket. Include a one-click "copy nudge" that produces a polite pre-written Teams message about the ticket.

- Cycle-time stage breakdown as stacked horizontal bars per ticket (one color per stage) — the single most persuasive visual in the app (answers C3 at a glance) — plus per-stage median bars for the team bottleneck view.
- Timeline view: vertical event stream (icons for transition / comment / worklog / assignee change), inactivity gaps rendered as visible spacers labeled “14 days — no activity”, stage-duration ribbon across the top.
- Meeting Mode toggle: aggregates only, names hidden, distributions instead of per-person rows, week-over-week deltas, large type for screen sharing (C11).
- Admin settings screen: drag statuses into classification buckets, per-status aging thresholds, checklist item configuration, comment-window and keyword lists, board IDs. This removes the deploy-per-workflow-change problem the document’s §7 warns about.

Data and trust details

- Freshness indicator on every page (“data as of 9:42”) with background refresh — changelog fetches are slow; never block a page on a live pull (the 5-minute cache already exists; surface it).
- Medians and p85 by default, averages never alone; show underlying counts next to every rate (a 50% return rate on 2 bugs is noise).
- Explain-this-number: hover any metric to see its definition and which statuses/rules feed it — crucial for trust, and the definitions already exist in the PRD glossary.
- Color plus icon (never color alone), sortable sticky-header tables, CSV/Excel export on every view (largely built), and a scheduled morning digest to a Teams channel via incoming webhook — the lightest path to daily adoption.
- Empty states that teach: the Sprint screen should say “No sprint boards configured — here is what to set up” rather than showing an empty table.

8. Suggested Build Sequence

Phase	Scope	Prerequisite
1 (1–2 wks)	Ingest comments + worklogs; agree status classification and surface it in an admin settings screen; ship My Day checklist + Attention Board (silent + aging reasons)	Decision 1
2 (1–2 wks)	QA Handoff screen (handoff feed, simplified quality checks, returned-from-QA with reasons); Ticket Investigator timeline UI	Comment ingestion
3 (1 wk)	Flow Analytics polish: stage-breakdown bars, medians/p85, Focus view; Quality screen (bug lens)	—
4 (parallel)	Process rollout: due-date + start-date rules, blocked convention (Flagged), Reopen-requires-comment; then add Overdue/slip and Blocked reasons to the Attention Board	Decisions 2–4
5 (when ready)	Sprint boards set up in Jira → flip on the already-coded Sprint Health; Team Trends layer + Meeting Mode; Teams webhook digest; SQLite	Decision 5

Phase	Scope	Prerequisite
	snapshots for week-over-week trends	

Phases 1–3 need no Jira process changes and deliver the daily-use surfaces. Phase 4 is where the process decisions land. Phase 5 completes the leadership layer. Everything builds on the engine already running in Azure.